

Faster printf Debugging (Part 1)

Nathan Jones

Contents

- [Introduction](#)
- [Establishing a Baseline](#)
- [Improve the Default Settings](#)
 - [Use a faster clock rate or processor](#)
 - [Use a faster baud rate](#)
 - [Ditch the HAL](#)
- [Circumvent the Problem](#)
 - [Make fewer calls to printf/snprintf](#)
 - [Simplify your calls to printf/snprintf](#)
 - [Shorten your messages](#)
 - [Simulate your system](#)
- [Conclusion](#)

Introduction

Debugging via `printf` (or `Serial.print` or, as I suggested in a previous article, `snprintf`) can be an extremely useful tool for seeing, quickly, what your system is doing and to zero in on the parts of your system that are, or could soon be, causing errors. This tool is not without its downsides, however, and you may have found in your experimentation with `printf` that calling it more than a few times **significantly slows down your program**. This challenge limits the times you can use `printf/snprintf` IRL, despite how useful it can be. In this article (and in Part 2) we'll do our best to **make `printf/snprintf` blisteringly fast** so that you can use it to your heart's content!

Establishing a Baseline

How much time does `printf` take *really*? To answer that, I'm going to execute the code below on a Nucleo-F042K6 running at 8 MHz.

```
HAL_GPIO_WritePin(A7_GPIO_Port, A7_Pin, GPIO_PIN_SET);  
printf("%s", "Whazzup, Nate Jones?!\\n");  
HAL_GPIO_WritePin(A7_GPIO_Port, A7_Pin, GPIO_PIN_RESET);
```

That whole process (which included sending out **22 characters** over UART) took about **6.8 ms**, as can be seen on the screenshot of my logic analyzer below. (About 145 μ s of that is `printf` formatting our message and 6.67 ms is the actual UART transmission.)

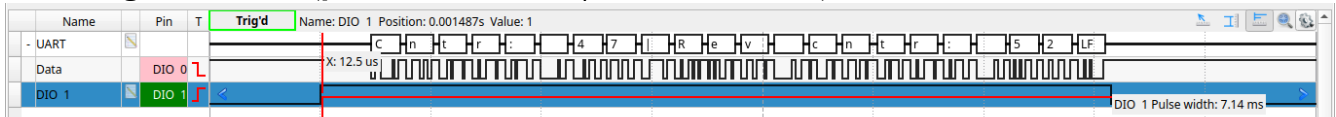


6.8 ms to send out *each one of our messages* is a long time!! No wonder our programs were slowing down so much. And, actually, it gets worse: that time was just for a message that had no variable arguments. Here are a few more tests that show how long `printf` takes if we pass in integer or floating-point numbers (with the same number of total characters).

One integer: 7.03 ms (`printf`: 0.365 ms | UART: 6.668 ms)



Two integers: 7.13 ms (`printf`: 0.470 ms | UART: 6.655 ms)



One float: 8.01 ms (`printf`: 1.333 ms | UART: 6.675 ms)



Two floats: 9.15 ms (`printf`: 2.475 ms | UART: 6.675 ms)



As it turns out, `printf` runs **non-deterministically**, meaning that it will take a *variable* amount of time to execute based on the number and type of variable arguments it's passed. It even depends on the specific *value* of those arguments sometimes, as certain values convert to strings more quickly than others! (Indeed, the length of time it took to send out the messages above, especially for the floating-point numbers, was highly variable from message to message.)

Maybe you actually *haven't* noticed the effect of `printf` on your system as its running, though. That's great! If all your system is doing is blinking an LED at 1 Hz, you can send out *books* of `printf` messages in between each LED blink. Eventually, though, as you add more `printf` statements and as your system does more and better things, you may realize that it *does* matter. If your actual system is only idle for 10 ms out of every 1 second then all of your print statements during that 1 second operating interval *have* to fit within that 10 ms budget or things will start to lag. Ultimately, this is a problem of **task scheduling**, i.e. ensuring that the tasks in your system, even with all their various print statements, can all complete before their respective deadlines. We need `printf` to run fast *enough* for our system to function, but not necessarily any faster. (For a more thorough discussion of task scheduling, see [here](#).)

Based on the timing diagrams above, we can group our solutions to the problem of making `printf`/`snprintf` faster into three broad categories:

1. **Circumvent the problem:** Change your approach or tactic so that `printf`/`snprintf` being slow matters less.

2. **Format faster:** Do something so that the time it takes `printf/snprintf` to format a message happens faster. These would reduce the times above that are 145 μ s – 2.475 ms.
3. **Transmit faster:** Do something so that the actual characters are transmitted to your computer faster. These would reduce the times above that are ~6.7 ms.

There are a number of different things to do in each category to meet our objectives. Some are harder than others, though, so look out for the following symbols to help you pick the right one for your project and skill level:



Improve the Default Settings

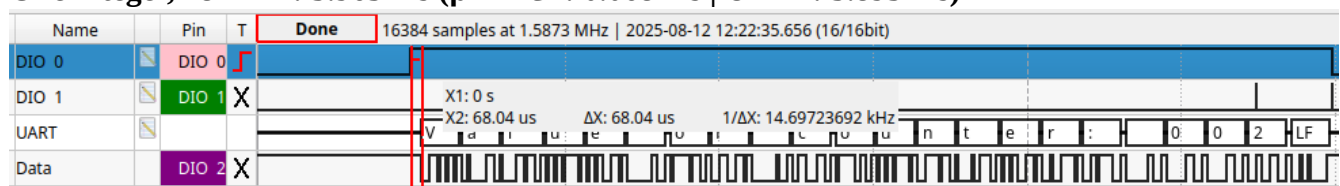
To begin we'll look at a few (mostly) easy optimizations we can make based simple on slightly changing the default settings for our project.

Use a faster clock rate or processor

Category	"Format Faster"			
Difficulty				
Time savings	1067 μ s			

An easy way to speed up our whole system is to simply run the clock faster. The STM32F042K6 I'm using in these examples can run as high as 48 Mhz, **six times faster** than the value that was used above (the default value for new STM32 projects for this Nucleo board)! At that clock speed, we can reduce the time it takes to format and send our message with one integer from 7.03 ms to only **5.96 ms!**

One integer, 48 MHz: 5.963 ms (`printf`: 0.068 ms | UART: 5.895 ms)



This significantly reduced the time it took for `printf` to run (from 365 μ s down to 68 μ s), though the time its taking to actually transmit our message is still dominating the execution time for the moment.

By a similar token, you could also move your project to a more powerful processor. This is possibly not the most helpful piece of advice, but it *does* bear mentioning. A faster microcontroller would not only reduce the time it took to format our debug messages and it would potentially allow our system to get more work done while each of our debug messages is being transmitted.

Use a faster baud rate

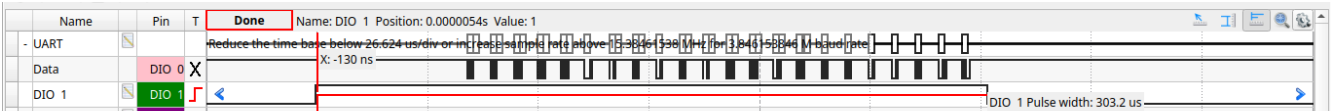
Category	"Transmit Faster"			
Difficulty				
Time savings	5660 μ s			

"Baud rate" is synonymous with "symbol rate" (which is how fast a communication channel can send individual symbols). In our case, that's the same as its "bit rate". If a UART channel, for instance, is set to a baud rate of 115200, it can send 115,200 bits per second. Since each byte we send has a total of 10 bits (which includes the start and stop bits), then we can send 11,520 bytes or characters per second across that UART link.

However, this likely isn't the maximum rate at which we can send data over UART! Many microcontrollers support baud rates as high as several **megabaud** and although, at some point, you'll need to worry about transmitting data so fast that it gets garbled in transmission, your UART link is likely short enough that setting it that high won't cause any problems (some experimentation required). If your UART is set to 38400 (the default value for new projects in STM32CubeIDE), increasing the baud rate to 3 Mbaud (the maximum that STM32CubeIDE will let me set it for my Nucleo-F042K6) is more than a **seventy-eightfold increase in speed**, meaning you can send nearly 30 times as many debug messages as before! (If your UART is, god forbid, still set to the Arduino default of 9600, this is over a three hundredfold increase in speed!)

Increasing the baud rate of our sample system from 38400 to 3M reduces the amount of time it takes to send our "one integer" message from 5.96 ms to **only 303 µs**! That's over 23 times faster than our initial baseline!

One integer, 48 MHz, 3 Mbaud: 0.303 ms (printf: 0.068 ms | UART: 0.235 ms)



Ditch the HAL

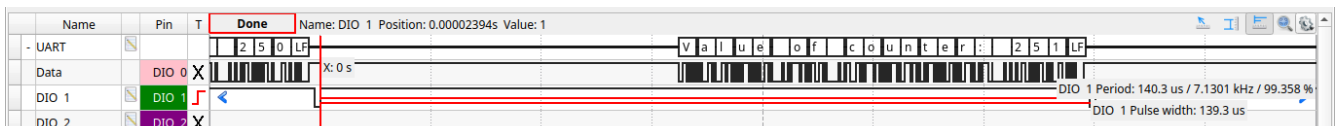
Category	"Transmit Faster"			
Difficulty				
Time savings	161 µs			

At this point I had the suspicion that the STM32 HAL was slowing me down a little bit. (HALs can be very convenient, but this convenience usually comes at the cost of lots of code.) I switched to using the STM32 LL ("low-level") library, which necessitated the following change to PUTCHAR_PROTOTYPE:

```
PUTCHAR_PROTOTYPE
{
    while(!LL_USART_IsActiveFlag_TXE(USART2));
    LL_USART_TransmitData8(USART2, ch);
    return ch;
}
```

The function HAL_UART_Transmit() was 85 lines of code, whereas LL_USART_TransmitData8() is 1 line of code, so it runs much more quickly. Switching to the low-level library reduced our message time from 303 µs to **140 µs**.

One integer, 48 MHz, 3 Mbaud, LL: 0.140 ms (printf: 0.065 ms | UART: 0.075 ms)



Circumvent the Problem

The next set of optimizations don't directly affect the time it takes to format or transmit our messages; instead, they focus on reducing the number and complexity of our messages to allow `printf/snprintf` to naturally complete more quickly. In the last tip, we may even just find that we don't need to optimize `printf/snprintf` at all!

Make fewer calls to `printf/snprintf`

Category	"Circumvent the Problem"			
Difficulty				
Time savings	Varies			

Instead of sending out individual values each time they occur in your system, you can consider sending out just a summary piece of data at a much lower frequency. For example:

- Instead of sending out each ADC value you measure, compute the **max**, **min**, **average**, and **standard deviation** of your ADC values and then send out those values once per second.
- Instead of sending out the execution time of a certain function *every* time it runs, log each execution time in a **histogram** (e.g. an array of values corresponding to execution times of, for example, [0-100 μ s, 100-500 μ s, 500-1000 μ s, 1000+ μ s]) and then transmit the *histogram* once per second.

This would add a little bit of processing that needed to be done on your microcontroller, but it could drastically reduce the number of messages you need to send.

Simplify your calls to `printf/snprintf`

Category	"Circumvent the Problem"			
Difficulty				
Time savings	Varies			

We learned above that `printf/snprintf` take longer when they are given more (and more complex) arguments, so consider reducing those to reduce the time spent formatting your debug messages. Find ways to:

- send out fewer overall arguments,
- require less pre-processing of the values you do send, and
- favor integers or fixed-point numbers over floats.

For example:

- Don't send out how far your robot traveled in what time *and* how fast its going, since how fast its going can be calculated later based on the first two pieces of data.
- Don't convert your ADC reading of the current temperature to Celsius and send that out, just send out the raw ADC reading since conversion to Celsius doesn't change and can be done later.
- Instead of operating on the *decimal* number of seconds since system startup (a floating-point number), try to operate on the *integer* number of *milliseconds*, since the integer will be faster for `printf` to format than the floating-point number.

Shorten your messages

Category	“Circumvent the Problem”			
Difficulty				
Time savings	Varies; ~0.5 μ s/ASCII char			

Every byte counts, so make those debug messages as terse as you can! Replacing `I bring you dire news, dear human, that the internal temperature, in Celsius, of the warp core has exceeded normal operating values and is now, even as I send you this message, rapidly approaching 39.4523 degrees!! Pray, take action, post haste!` with `WARNING: Warp core 39.4523 deg C` saves 213 characters, which is 213 fewer characters that `printf/snprintf` have to process before the message is sent out.

Okay, so your debug messages probably aren't that verbose (or as fun), but the point stands. Even replacing `WARNING: Warp core 39.4523 deg C` with `[W] Core 39C` saves 20 characters and is nearly one-third the length of the former message. (Let's be honest: there's probably only one core [no need to specify "warp"] and you probably don't need that much precision in the temperature.)

On my Nucleo-F042K6, changing `Value of counter: %03d\n` to just `Counter:%03d\n` (saving 11 characters) reduced the formatting time by 6 μ s, or roughly **half a microsecond per ASCII character**.

Simulate your system

Category	“Circumvent the Problem”			
Difficulty				
Time savings	N/A			



Why get dirty with the messiness of reality when you can run your whole system inside your computer instead?? With a simulation, the amount of time `printf/snprintf` takes is less important, owing to the facts that

- we have a much more powerful processor executing our program (so `printf/snprintf` runs faster almost automatically),
- we're mostly concerned with functional simulation over timing-accurate simulation, and
- we have more control over when and in what order our system events arrive.

To do this you would replace all of the hardware-specific code on your embedded system with versions that could run on your computer instead. It's like setting up a "mock" for a unit test: you're simply writing code that is *pretending* to act like an external device, but it gets to do that in any way it wants as long as it *behaves* like the real thing. For example:

- Instead of reading time with an RTC over SPI, you can call `std::chrono::system_clock::now()`.
- Instead of reading the analog value of a potentiometer, you can prompt the computer user for a value or read values from a file.
- Instead of controlling the speed of a motor, you can simply use `printf` to display its updated speed.
- etc

Here's what this might look like for a system that was reading accelerometer values and setting the speed of a motor based on those values (a full description of what these videos are doing and how to program them yourself can be found [here](#)). In the first example I'm getting user inputs (or generating random numbers) to simulate my sensors and just writing motor values to the terminal. In the second example, I'm doing the same thing in a much fancier way by getting and displaying values in a GUI.

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/erZ2x05wkJA?
si=c0tkYfzrF5I_SSs_" title="YouTube video player" frameborder="0" allow="accelerometer; autoplay;
clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share" referrerpolicy="strict-
origin-when-cross-origin" allowfullscreen="">
</iframe>
```

```
<p></p>
```

<iframe width="560" height="315" src="https://www.youtube.com/embed/c46W2OUhtFo?si=DRMhIenLUIQJNm2N" title="YouTube video player" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share" referrerpolicy="strict-origin-when-cross-origin" allowfullscreen="">
</iframe>

You might find that you don't actually *need* `printf` to run faster, as long as it can run in a simulation that faithfully shows you what your program *would* do on a real device.

Conclusion

With only a few easy modifications, we were able to make significant improvements in how fast `printf/snprintf` ran!

Example	Clock (MHz)	Baud rate	HAL / LL	Formatting (µs)	Transmitting (µs)	Total time ¹ (µs)
Baseline	8	38400	HAL	365	6,668	7,030
Faster clock	48	38400	HAL	68	5,895	5,963
Faster baud	48	3000000	HAL	68	235	303
Ditch the HAL	48	3000000	LL	65	75	140

1: Time to send a 22 character message with one integer

By just clicking a few buttons in our setup and changing one line of code, we made `printf` more than **50 times faster!**

Additionally, we identified a few ways you can use `printf/snprintf` less overall, further reducing the amount of time your processor is being used to send out messages.

If you've made it this far, thanks for reading and happy hacking!